Proiectarea și Analiza Algoritmilor

Laborator – Patru probleme aplicative

Universitatea Politehnica Timișoara

Departamentul de Calculatoare și Tehnologia Informației

Anul II – Curs Aplicativ

Cuprins

1. Problema 1 – Catalogul unei farmacii
2. Problema 2 – Tastatura predictivă a unei aplicații
3. Problema 3 – Rețea de fibră optică în campus
4. Problema 4 – Romeo și Julieta în Verona

Problema 1 – Enunț

Context

O farmacie ține evidența loturilor de medicamente primite. Fiecare lot are un *cod unic* (număr întreg pozitiv, nu neapărat consecutiv), iar farmacistul are nevoie de o structură care să răspundă rapid la următoarele operații, indiferent câte mii de loturi sunt în catalog: înregistrarea unui lot nou la sosire, eliminarea unui lot atunci când s-a epuizat, verificarea existenței unui cod, găsirea celui mai apropiat cod mai mare decât un cod dat (pentru când lotul cerut nu este disponibil), și listarea tuturor codurilor în ordine crescătoare pentru raportul zilnic.

Obiectiv

Toate operațiile de mai sus trebuie să se execute, în medie, în timp proporțional cu *logaritmul* numărului de loturi, nu cu numărul de loturi.
Studenții trebuie să identifice singuri structura de date potrivită și să o implementeze de la zero – nu se admit biblioteci de tip `set`/`map`.

Problema 1 – Cerințe

C1. Construiți structura, plecând de la o secvență inițială de inserări. Implementați operația de inserare a unui cod nou și operația de căutare a unui cod existent.

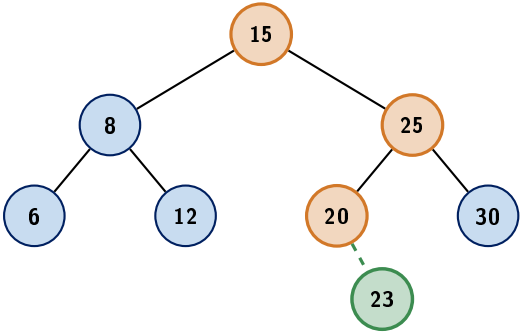
C2. Implementați operația de ștergere a unui cod (tratând corect toate cazurile particulare: cod inexistent, frunză, nod cu un singur descendent, nod cu doi descendenți).

C3. Implementați: codul minim, codul maxim, *succesorul* unui cod dat (cel mai mic cod strict mai mare decât valoarea de interogare), precum și listarea tuturor codurilor în ordine crescătoare.

Date de intrare

Un șir de operații, fiecare de forma: `INSERT x`, `DELETE x`, `FIND x`, `SUCC x`, `LIST`, `MIN`, `MAX`.

Problema 1 – Exemplu vizual



Inserare lot nou: cod 23

Drumul de căutare (portocaliu):
$15 \rightarrow 25 \rightarrow 20$
$20 < 23$, deci pasul următor ar fi în dreapta lui 20. Nodul nu există $\Rightarrow$ se creează aici (verde).

Succesor(19): cel mai mic cod > 19 este 20.
Listare: 6, 8, 12, 15, 20, 23, 25, 30.

C1 – Construire și căutare

Cerința

Implementați inserarea unui cod nou și căutarea unui cod existent.

Indicii / discuție la tablă:

- Dacă fiecare nod ar memora un cod și am vrea să găsim un cod căutat folosind o singură comparație pe nivel, cum ar trebui aranjate codurile între părinte și descendenți?
- De ce această aranjare aduce căutarea la $O(\log n)$ în medie? Când degenerează la $O(n)$?
- Cum arată inserarea unui cod nou? Unde îl plasăm pentru a păstra proprietatea cerută la căutare?

C1 – Pseudocod

Idee: Fiecare nod conține un cod și două subarbori: unul cu toate codurile strict mai mici, altul cu toate codurile strict mai mari. Atunci căutarea poate elimina jumătate din arbore la fiecare pas (asemănător cu căutarea binară într-un șir sortat).

Listing 1: Pseudocod – inserare și căutare

```
insert(t, x):
    daca t == NULL: returneaza nod_nou(x)
    daca x < t.cod: t.stanga = insert(t.stanga, x)
    altfel daca x > t.cod: t.dreapta = insert(t.dreapta, x)
    // daca x == t.cod: cod duplicat, ignoram
    returneaza t

find(t, x):
    daca t == NULL: returneaza NULL
    daca x == t.cod: returneaza t
    daca x < t.cod: returneaza find(t.stanga, x)
    altfel: returneaza find(t.dreapta, x)
```

Complexitate: $O(h)$ per operație, unde h este înălțimea structurii. Pentru intrări tipice (aleatoare), $h \approx \log_2 n$.

C1 – Implementare în C

```c
typedef struct Nod {
    int cod;
    struct Nod *stanga, *dreapta;
} Nod;

Nod* nod_nou(int x) {
    Nod *n = (Nod*) malloc(sizeof(Nod));
    n->cod = x;
    n->stanga = n->dreapta = NULL;
    return n;
}

Nod* insert(Nod *t, int x) {
    if (t == NULL) return nod_nou(x);
    if      (x < t->cod) t->stanga  = insert(t->stanga,  x);
    else if (x > t->cod) t->dreapta = insert(t->dreapta, x);
    return t;
}

Nod* find(Nod *t, int x) {
    if (t == NULL || t->cod == x) return t;
    if (x < t->cod) return find(t->stanga,  x);
    else            return find(t->dreapta, x);
}
```

C2 – Ștergerea unui cod

Cerința

Implementați operația de ștergere a unui cod, tratând corect toate cazurile particulare.

Indicii / discuție la tablă:

- Câte cazuri distincte apar la ștergere? Discutați separat: nodul nu există, nodul este frunză, nodul are un singur descendent, nodul are doi descendenți.
- Pentru cazul cu doi descendenți: cu ce cod îl putem înlocui astfel încât proprietatea de ordine să fie păstrată? Care sunt cele două opțiuni clasice?
- De ce nu putem pur și simplu să copiem codul fiului stâng sau drept în locul celui șters?

C2 – Pseudocod

Idee: Pentru cazul cu doi descendenți, înlocuim codul nodului cu cel mai mic cod din subarborele drept (*succesorul in-order*), apoi ștergem recursiv acel cod din subarborele drept (unde devine o problemă cu cel mult un descendent).

Listing 2: Pseudocod – ștergere

```
delete(t, x):
    daca t == NULL: returneaza NULL
    daca x < t.cod: t.stanga  = delete(t.stanga,  x)
    altfel daca x > t.cod: t.dreapta = delete(t.dreapta, x)
    altfel:                          // am gasit nodul de sters
        daca t.stanga == NULL:  returneaza t.dreapta
        daca t.dreapta == NULL: returneaza t.stanga
        s = min_din_subarbore(t.dreapta)  // succesor
        t.cod = s.cod
        t.dreapta = delete(t.dreapta, s.cod)
    returneaza t

min_din_subarbore(t):
    cat timp t.stanga != NULL: t = t.stanga
    returneaza t
```

Complexitate: $O(h)$.

C2 – Implementare în C

```
Nod* min_din_subarbore(Nod *t) {
    while (t->stanga != NULL) t = t->stanga;
    return t;
}

Nod* delete(Nod *t, int x) {
    if (t == NULL) return NULL;
    if (x < t->cod) {
        t->stanga = delete(t->stanga, x);
    } else if (x > t->cod) {
        t->dreapta = delete(t->dreapta, x);
    } else {
        // nodul de sters
        if (t->stanga == NULL) {
            Nod *aux = t->dreapta;
            free(t);
            return aux;
        }
        if (t->dreapta == NULL) {
            Nod *aux = t->stanga;
            free(t);
            return aux;
        }
        // doi descendenti: ia succesorul in-order
        Nod *s = min_din_subarbore(t->dreapta);
        t->cod = s->cod;
        t->dreapta = delete(t->dreapta, s->cod);
    }
    return t;
}
```

C3 – Min, max, succesor, listare ordonată

Cerința

Implementați: codul minim, codul maxim, succesorul unui cod (cel mai mic cod strict mai mare), și listarea în ordine crescătoare a tuturor codurilor.

Indicii / discuție la tablă:

- În ce direcție trebuie să coborâm de la rădăcină pentru a ajunge la codul minim? Dar la cel maxim?
- Pentru succesorul lui x: ce facem când x are subarbore drept? Ce facem când nu are?
- Cum obținem listarea în ordine crescătoare *vizitând fiecare nod exact o singură dată*? În ce ordine vizităm: stâng, rădăcină, drept – sau o altă combinație?

Listing 3: Pseudocod

```
min(t):  cat timp t.stanga  != NULL: t = t.stanga;  returneaza t.cod
max(t):  cat timp t.dreapta != NULL: t = t.dreapta; returneaza t.cod

succesor(t, x):
    // gasim mai intai nodul x si in timpul coborarii
    // memoram cel mai recent "stramos" fata de care am virat la stanga
    candidat = NULL
    n = t
    cat timp n != NULL:
        daca x < n.cod:
            candidat = n            // pas posibil pentru succesor
            n = n.stanga
        altfel:
            n = n.dreapta
    daca candidat == NULL: returneaza "nu exista"
    returneaza candidat.cod

inorder(t):
    daca t == NULL: returneaza
    inorder(t.stanga)
    afiseaza t.cod
    inorder(t.dreapta)
```

Complexitate: $O(h)$ pentru min/max/succesor; $O(n)$ pentru listare.

C3 – Implementare în C

```c
int minim(Nod *t) {
    while (t->stanga != NULL) t = t->stanga;
    return t->cod;
}

int maxim(Nod *t) {
    while (t->dreapta != NULL) t = t->dreapta;
    return t->cod;
}

int succesor(Nod *t, int x, int *out) {
    Nod *candidat = NULL;
    while (t != NULL) {
        if (x < t->cod) { candidat = t; t = t->stanga; }
        else            {               t = t->dreapta; }
    }
    if (candidat == NULL) return 0;        // nu exista
    *out = candidat->cod;
    return 1;
}

void inorder(Nod *t) {
    if (t == NULL) return;
    inorder(t->stanga);
    printf("%d ", t->cod);
    inorder(t->dreapta);
}
```

Problema 2 – Enunț

Context

O aplicație de mesagerie include o tastatură predictivă care învață din cuvintele tastate de utilizator. Pentru fiecare cuvânt scris, aplicația îi crește un contor de frecvență. Când utilizatorul începe să tasteze un cuvânt nou și ajunge la un prefix de câteva litere, tastatura trebuie să sugereze cuvântul *cel mai frecvent* care începe cu acel prefix, precum și să poată lista *toate* cuvintele care încep cu acel prefix.

Obiectiv

Operațiile de înregistrare / actualizare a unui cuvânt, și de căutare a sugestiei pentru un prefix, trebuie să se execute în timp proporțional cu *lungimea cuvântului sau a prefixului*, indiferent de câte mii de cuvinte are vocabularul utilizatorului. Cuvintele sunt formate doar din litere mici ale alfabetului englez (26 de litere).

Problema 2 – Cerințe

C1. Construiți structura de date. Implementați operația de adăugare a unui cuvânt nou sau de incrementare a frecvenței unui cuvânt deja existent.

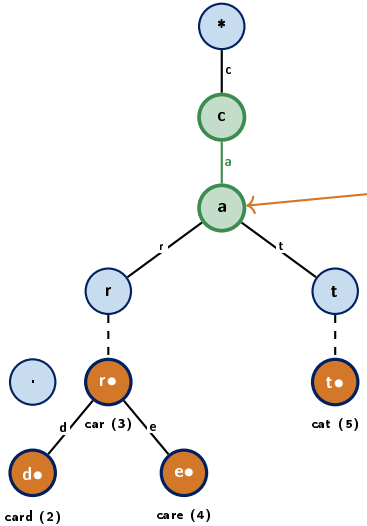
C2. Pentru un prefix dat, listați *toate* cuvintele din vocabular care încep cu acel prefix (în orice ordine).

C3. Pentru un prefix dat, returnați *cuvântul cel mai frecvent* care începe cu acel prefix. Această interogare trebuie să fie $O(|\text{prefix}|)$, nu $O(|\text{prefix}| + \text{numărul de cuvinte cu prefixul!})$.

Date de intrare

Un șir de operații: `ADD cuvant` (adăugare / incrementare frecvență), `LIST prefix`, `TOP prefix`.

Problema 2 – Exemplu vizual



Nod prefix "ca":
best = "cat" (5)

Vocabular: cat(5), car(3), card(2), care(4)

TOP("ca") $\rightarrow$ "cat"

LIST("car") $\rightarrow$ {car, card, care}

C1 – Adăugare / actualizare cuvânt

Cerința

Construiți structura. Implementați adăugarea unui cuvânt nou sau incrementarea frecvenței unui cuvânt existent.

Indicii / discuție la tablă:

- Dacă fiecare nod ar reprezenta un prefix, ce informație ar conține arcele dintre noduri? Câte arce poate avea, cel mult, un nod?
- Cum reprezentăm faptul că un nod marchează *sfârșitul* unui cuvânt (nu doar un prefix intermediar)?
- Pentru cerința C3 (cuvântul cel mai frecvent cu prefixul dat), de ce nu putem amâna până la momentul interogării? Ce informație trebuie să menținem *în noduri* pe parcursul inserărilor?

C1 – Pseudocod

Idee: Fiecare nod are 26 de copii (câte unul pentru fiecare literă) și marchează un prefix. Un cuvânt este reprezentat prin drumul de la rădăcină până la un nod marcat ca *sfârșit de cuvânt*. Pentru C3, fiecare nod va memora *cel mai bun cuvânt* din subarborele său.

Listing 4: Pseudocod – adăugare

```
add_word(root, w):
    drum = [root]                    // memoram drumul parcurs
    cur = root
    pentru fiecare litera c din w:
        daca cur.copii[c] == NULL:
            cur.copii[c] = nod_nou()
        cur = cur.copii[c]
        adauga cur la drum
    // cur este nodul de final pentru cuvantul w
    cur.este_sfarsit = 1
    cur.id_cuvant = id_in_dictionar(w)   // memoreaza w in dictionar global
    cur.frecventa = cur.frecventa + 1

    // actualizeaza "cel mai bun" pe tot drumul
    pentru fiecare n din drum:
        daca n.best_frecv < cur.frecventa:
            n.best_frecv  = cur.frecventa
            n.best_id     = cur.id_cuvant
```

C1 – Implementare în C

```c
#define ALF 26
#define MAXW 5000
#define MAXL 64

typedef struct Trie {
    struct Trie *copii[ALF];
    int este_sfarsit;
    int id_cuvant;        // -1 daca nu este sfarsit de cuvant
    int frecventa;        // frecventa cuvantului care se termina aici
    int best_id;          // id-ul celui mai bun cuvant din subarbore
    int best_frecv;       // frecventa celui mai bun cuvant
} Trie;

char dictionar[MAXW][MAXL];
int  dict_size = 0;

Trie* nod_nou(void) {
    Trie *t = (Trie*) calloc(1, sizeof(Trie));
    t->id_cuvant = -1;
    t->best_id   = -1;
    return t;
}

int id_in_dictionar(const char *w) {
    for (int i = 0; i < dict_size; i++)
        if (strcmp(dictionar[i], w) == 0) return i;
    strcpy(dictionar[dict_size], w);
    return dict_size++;
}
```

C1 – Implementare în C (cont.)

```c
void add_word(Trie *root, const char *w) {
    Trie *drum[MAXL + 1];
    int  drum_len = 0;
    drum[drum_len++] = root;

    Trie *cur = root;
    for (int i = 0; w[i] != '\0'; i++) {
        int c = w[i] - 'a';
        if (cur->copii[c] == NULL) cur->copii[c] = nod_nou();
        cur = cur->copii[c];
        drum[drum_len++] = cur;
    }
    // marcheaza sfarsit de cuvant si incrementeaza frecventa
    if (!cur->este_sfarsit) {
        cur->este_sfarsit = 1;
        cur->id_cuvant    = id_in_dictionar(w);
    }
    cur->frecventa++;

    // actualizeaza "cel mai bun" pe tot drumul de la radacina pana aici
    for (int i = 0; i < drum_len; i++) {
        if (drum[i]->best_frecv < cur->frecventa) {
            drum[i]->best_frecv = cur->frecventa;
            drum[i]->best_id    = cur->id_cuvant;
        }
    }
}
```

C2 – Listare toate cuvintele cu un prefix dat

Cerința

Pentru un prefix dat, afișați toate cuvintele din vocabular care încep cu acel prefix.

Indicii / discuție la tablă:

- Cum "ajungem" la nodul corespunzător prefixului? Câte comparații sunt necesare?
- Ce trebuie făcut dacă, pe parcursul coborârii pentru prefix, un copil nu există?
- Odată ajunși la nodul-prefix, cum enumerăm *toate* cuvintele din subarborele său? Cum reconstruim fiecare cuvânt pe parcursul enumerării?

C2 – Pseudocod

Idee: Coborâm de la rădăcină urmând literele prefixului. Dacă pe parcurs un copil lipsește, nu există cuvinte cu acel prefix. Altfel, din nodul-prefix facem o parcurgere recursivă (DFS) care construiește cuvintele literă cu literă într-un buffer, afișând-le la fiecare nod marcat ca sfârșit de cuvânt.

Listing 5: Pseudocod

```
list_words(root, prefix):
    cur = root
    pentru fiecare litera c din prefix:
        daca cur.copii[c] == NULL: returneaza "nu exista"
        cur = cur.copii[c]
    // cur este nodul-prefix
    buf = copie a lui prefix
    dfs_collect(cur, buf, lungime(prefix))

dfs_collect(n, buf, len):
    daca n.este_sfarsit:
        afiseaza buf[0..len-1]
    pentru fiecare litera c in 0..25:
        daca n.copii[c] != NULL:
            buf[len] = 'a' + c
            dfs_collect(n.copii[c], buf, len + 1)
```

C2 – Implementare în C

```c
void dfs_collect(Trie *n, char *buf, int len) {
    if (n->este_sfarsit) {
        buf[len] = '\0';
        printf("  %s  (frecv = %d)\n", buf, n->frecventa);
    }
    for (int c = 0; c < ALF; c++) {
        if (n->copii[c] != NULL) {
            buf[len] = 'a' + c;
            dfs_collect(n->copii[c], buf, len + 1);
        }
    }
}

void list_words(Trie *root, const char *prefix) {
    Trie *cur = root;
    for (int i = 0; prefix[i] != '\0'; i++) {
        int c = prefix[i] - 'a';
        if (cur->copii[c] == NULL) {
            printf("(nu exista cuvinte cu prefixul '%s')\n", prefix);
            return;
        }
        cur = cur->copii[c];
    }
    char buf[MAXL];
    strcpy(buf, prefix);
    dfs_collect(cur, buf, strlen(prefix));
}
```

C3 – Cuvântul cel mai frecvent cu un prefix dat

Cerința

Pentru un prefix dat, returnați cuvântul cu cea mai mare frecvență care începe cu acel prefix. Operația trebuie să se execute în $O(|\text{prefix}|)$.

Indicii / discuție la tablă:

- Pentru ca răspunsul să fie în $O(|\text{prefix}|)$, ce trebuie să avem deja pregătit la *fiecare* nod al structurii?
- În C1 am întreținut, la fiecare nod de pe drumul inserării, "cel mai bun cuvânt" din subarbore. Cum folosim această informație acum?
- Ce returnăm dacă prefixul nu există în structură?

C3 – Pseudocod și implementare în C

Idee: Coborâm până la nodul-prefix. Acolo, câmpul `best_id` (întreținut la inserări) ne dă imediat răspunsul.

Listing 6: Pseudocod

```
top_completion(root, prefix):
    cur = root
    pentru fiecare litera c din prefix:
        daca cur.copii[c] == NULL: returneaza "(nu exista)"
        cur = cur.copii[c]
    daca cur.best_id < 0: returneaza "(nu exista)"
    returneaza dictionar[cur.best_id]
```

```
const char* top_completion(Trie *root, const char *prefix) {
    Trie *cur = root;
    for (int i = 0; prefix[i] != '\0'; i++) {
        int c = prefix[i] - 'a';
        if (cur->copii[c] == NULL) return NULL;
        cur = cur->copii[c];
    }
    if (cur->best_id < 0) return NULL;
    return dictionar[cur->best_id];
}
```

Problema 3 – Enunț

Context

Administrația unei universități vrea să tragă fibră optică între cele n clădiri ale campusului. Compania de telecomunicații a inspectat campusul și a furnizat o listă de m trasee directe posibile, fiecare cu un cost de instalare în euro: (u_i, v_i, c_i), unde u_i și v_i sunt clădirile, iar $c_i \geq 0$ este costul. Două clădiri pot fi conectate fie direct, fie indirect printr-un lanț de clădiri intermediare.

Obiectiv

Alegeți un subansamblu de trasee astfel încât toate clădirile să fie conectate (direct sau indirect) între ele, iar costul total al instalării să fie *minim*. În plus, administrația dorește să cunoască "traseul-strangulare" – cel mai scump traseu din rețeaua aleasă, pentru a-l negocia în viitor cu compania.

Problema 3 – Cerințe

C1. Calculați costul minim total al rețelei și verificați dacă proiectul este realizabil (toate clădirile pot fi conectate cu traseele disponibile).

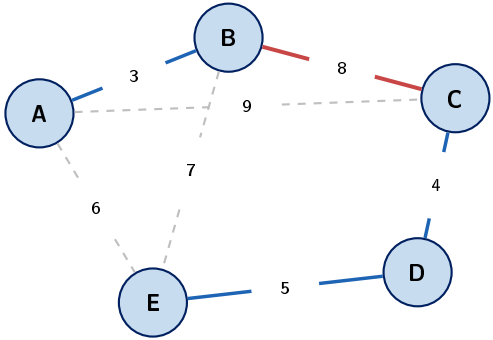
C2. Afișați lista efectivă a traseelor selectate (în ordinea în care au fost alese de algoritm).

C3. Identificați *traseul-strangulare*: cablul cu cel mai mare cost din rețeaua aleasă. Argumentați de ce, pentru un proiect cu cost minim, acest traseu este *neapărat* cel mai scurt dintre toate cele care ar conecta cele două "grupuri" de clădiri pe care le unește.

Date de intrare

`n`, `m`, urmate de `m` triplete (u_i, v_i, c_i).

Problema 3 – Exemplu vizual



trasee alese (cost normal)
traseul-strangulare (max)
trasee neutilizate

Cost total: $3 + 4 + 5 + 8 = 20$
Strangulare: B–C (8)

C1 – Cost minim și verificare realizabilitate

Cerința

Calculați costul minim total și verificați dacă proiectul este realizabil.

Indicii / discuție la tablă:

- Dacă sortăm traseele crescător după cost și le parcurgem în ordine, când este sigur să adăugăm un traseu și când îl sărim?
- Cum verificăm rapid dacă două clădiri sunt deja conectate (direct sau indirect) prin traseele alese până acum? Ce structură de date permite *unirea* a două grupuri și *verificarea apartenenței* aproape în timp constant?
- Câte trasee selectează algoritmul dacă proiectul este realizabil? Ce indică o valoare mai mică decât aceasta?

C1 – Pseudocod

Idee: Sortăm traseele crescător după cost. Pentru fiecare în ordine, îl adăugăm dacă *nu creează un ciclu* (adică nu conectează două clădiri deja conectate). Folosim o structură de tip Union-Find: `find(x)` returnează un reprezentant al grupului din care face parte x, iar `union(x, y)` unește cele două grupuri.

Listing 7: Pseudocod

```
sort trasee crescator dupa cost
init Union-Find cu n clase (cate o cladire fiecare)
cost_total = 0; nr_alese = 0
pentru fiecare traseu (u, v, c) in ordine:
    daca find(u) != find(v):
        union(u, v)
        cost_total = cost_total + c
        nr_alese  = nr_alese + 1
        daca nr_alese == n - 1: oprire
daca nr_alese < n - 1: proiect nerealizabil
altfel: afiseaza cost_total
```

Complexitate: $O(m \log m)$ (domină sortarea); operațiile Union-Find sunt aproape $O(1)$ amortizat cu compresie + rang.

C1 – Implementare în C (Union-Find)

```c
typedef struct { int u, v, c; } Traseu;

int parinte[NMAX], rang[NMAX];

void uf_init(int n) {
    for (int i = 1; i <= n; i++) { parinte[i] = i; rang[i] = 0; }
}

int find(int x) {
    while (parinte[x] != x) {
        parinte[x] = parinte[parinte[x]];   // compresie
        x = parinte[x];
    }
    return x;
}

int uf_union(int a, int b) {
    int ra = find(a), rb = find(b);
    if (ra == rb) return 0;              // deja in aceeasi clasa
    if      (rang[ra] < rang[rb]) parinte[ra] = rb;
    else if (rang[ra] > rang[rb]) parinte[rb] = ra;
    else                          { parinte[rb] = ra; rang[ra]++; }
    return 1;
}

int cmp(const void *a, const void *b) {
    return ((Traseu*)a)->c - ((Traseu*)b)->c;
}
```

C1 – Implementare în C (algoritm principal)

```c
typedef struct {
    long cost_total;
    int  realizabila;        // 1 daca da, 0 daca nu
    int  componente_ramase; // util pentru raportare
} Rezultat;

Rezultat retea_minima(Traseu *T, int m, int n) {
    qsort(T, m, sizeof(Traseu), cmp);
    uf_init(n);

    Rezultat r = {0, 1, 1};
    int nr_alese = 0;
    for (int i = 0; i < m && nr_alese < n - 1; i++) {
        if (uf_union(T[i].u, T[i].v)) {
            r.cost_total += T[i].c;
            nr_alese++;
        }
    }
    if (nr_alese < n - 1) {
        r.realizabila = 0;
        r.componente_ramase = n - nr_alese;
    }
    return r;
}
```

C2 – Lista traseelor selectate

Cerința

Afișați lista traseelor instalate, în ordinea în care au fost alese de algoritm.

Indicii / discuție la tablă:

- Ce mică modificare aducem algoritmului de la C1 pentru a reține nu doar costul, ci și *care* trasee au fost selectate?
- Ordinea în care le alegem corespunde cu ordinea crescătoare a costurilor? De ce?

C2 – Pseudocod și implementare

Idee: Reținem traseele acceptate de `uf_union` într-un vector.

```
int alege_trasee(Traseu *T, int m, int n,
                 Traseu *alese, long *cost_out) {
    qsort(T, m, sizeof(Traseu), cmp);
    uf_init(n);
    int nr = 0;
    long cost = 0;
    for (int i = 0; i < m && nr < n - 1; i++) {
        if (uf_union(T[i].u, T[i].v)) {
            alese[nr++] = T[i];
            cost += T[i].c;
        }
    }
    *cost_out = cost;
    return nr;          // numar de trasee alese
}

void afiseaza_trasee(Traseu *alese, int nr) {
    printf("Trasee instalate (in ordinea alegerii):\n");
    for (int i = 0; i < nr; i++)
        printf("  %3d -- %-3d  cost = %d\n",
               alese[i].u, alese[i].v, alese[i].c);
}
```

C3 – Traseul-strangulare

Cerința

Identificați traseul cu cel mai mare cost din rețeaua aleasă (“strangularea”) și explicați de ce este, în mod necesar, cel mai mic cost posibil pentru a uni cele două grupuri pe care le conectează.

Indicii / discuție la tablă:

- Având deja lista traseelor alese (de la C2), cum extragem rapid traseul-strangulare?
- **Argumentul cheie**: când algoritmul ajunge să adauge traseul cu cost maxim din rețea, ce stare are structura Union-Find? Ce ne spune asta despre toate celelalte trasee *neprocesate* încă (care au cost cel puțin egal)?
- Ar putea exista un alt arbore de cost minim cu o “strangulare” mai mică decât a noastră? Argument: dacă da, am putea înlocui traseul nostru maxim cu acela și am obține cost *strict* mai mic – contradicție.

C3 – Pseudocod și implementare

Idee: Deoarece traseele au fost sortate crescător, ultimul traseu adăugat de algoritm este chiar traseul-strangulare. Putem fie să-l reținem pe parcurs, fie să-l recuperăm la final din vectorul `alese`.

```
Traseu strangulare(Traseu *alese, int nr) {
    Traseu max = alese[0];
    for (int i = 1; i < nr; i++)
        if (alese[i].c > max.c) max = alese[i];
    return max;
}

// Folosire: dupa alege_trasee, apelam strangulare(alese, nr)
//    Traseu s = strangulare(alese, nr);
//    printf("Strangulare: %d -- %d  cost = %d\n", s.u, s.v, s.c);
```

Observație: Pentru o variantă elegantă, traseul-strangulare este pur și simplu `alese[nr - 1]`, deoarece traseele au fost alese în ordine crescătoare a costului.

Problema 4 – Enunț

Context

Verona are n piațete numerotate $1 \ldots n$, legate prin m alei orientate. Fiecare alee (u, v, t) poate fi parcursă *doar* de la u la v și are un timp de parcurgere t în minute. Timpii pot fi *negativi*: pe unele alei coborâte la vale călătorul "câștigă timp" față de mersul normal (e mai rapid decât ar zice modelul liniar), iar pe altele se urcă în deal și timpul este pozitiv. Se garantează că rețeaua poate sau nu să conțină "cicluri suspecte" în care timpul total parcurs să fie negativ – aceasta ar însemna că trasarea cronografului este eronată.

Obiectiv

Romeo (în piațeta R) și Julieta (în piațeta J) doresc să se întâlnească cât mai repede posibil. Ambii se mișcă *simultan* prin rețea. Trebuie să alegem piațeta de întâlnire M astfel încât *maximul* dintre timpul lui Romeo (de la R la M) și timpul Julietei (de la J la M) să fie minim. În același timp, trebuie să detectăm dacă rețeaua este coerentă (fără cicluri cu timp total negativ).

Problema 4 – Cerințe

C1. Calculați, pentru toate perechile (i, j) de piațete, timpul minim de la i la j (sau $+\infty$ dacă j nu este accesibil din i). Algoritmul trebuie să suporte timpi negativi pe alei.

C2. Detectați dacă rețeaua conține un *ciclu cu timp total negativ*. Dacă da, raportați o piațetă care face parte dintr-un astfel de ciclu (analiza C3-C4 este invalidă în acest caz).

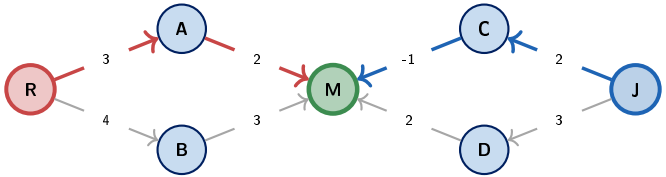
C3. Dacă rețeaua este coerentă, găsiți piațeta de întâlnire M care minimizează $\max(d(R, M), d(J, M))$, precum și timpul efectiv de întâlnire.

C4. Reconstruiți traseul efectiv urmat de Romeo (de la R la M) și cel al Julietei (de la J la M).

Date de intrare

`n`, `m`, `R`, `J`, urmate de `m` triplete (u_i, v_i, t_i).

Problema 4 – Exemplu vizual



Drumul lui Romeo: $R \to A \to M$, timp $= 5$. Drumul Julietei: $J \to C \to M$, timp $= 2 + (-1) = 1$.

Timp de întâlnire $= \max(5, 1) = 5$.

C1 – Drumurile minime pentru toate perechile

Cerința

Calculați $d[i][j]$ = timpul minim de la i la j pentru toate perechile, suportând timpi negativi.

Indicii / discuție la tablă:

- Pentru drum minim cu o singură sursă și *timpi nenegativi*, există un algoritm clasic. De ce nu mai funcționează dacă admitem timpi negativi?
- Putem aborda problema *global*, calculând odată drumurile minime între toate perechile? Care este ideea: dacă deja știm cel mai bun drum de la i la j care folosește doar piațete intermediare din mulțimea $\{1, \ldots, k-1\}$, cum îl putem "rafina" acceptând și piațeta k?
- Care este complexitatea acestei abordări? Este acceptabilă pentru n de ordinul sutelor?

C1 – Pseudocod

Idee (Floyd-Warshall): Considerăm rând pe rând fiecare piațetă k ca posibilă piațetă intermediară. Pentru fiecare pereche (i, j), drumul minim folosind intermediari din $\{1, \ldots, k\}$ este fie cel anterior (fără k), fie unul care trece prin k.

Listing 8: Pseudocod

```
1  init d[i][j] = +INF pentru toate perechile
2  init d[i][i] = 0
3  pentru fiecare alee (u, v, t):
4      d[u][v] = min(d[u][v], t)
5
6  pentru k = 1 .. n:
7      pentru i = 1 .. n:
8          pentru j = 1 .. n:
9              daca d[i][k] + d[k][j] < d[i][j]:
0                  d[i][j] = d[i][k] + d[k][j]
```

Complexitate: $T(n) = O(n^3)$, $S(n) = O(n^2)$. Funcționează corect și cu timpi negativi, atâta vreme cât nu există cicluri cu timp total negativ.

Atenție: folosim o constantă $+\infty$ "moderată" (de exemplu 10^9) pentru a evita depășirea la `d[i][k] + d[k][j]`.

C1 – Implementare în C

```c
#define INF 1000000000          // suficient de mare, dar evita overflow

long d[NMAX][NMAX];

void floyd(int n) {
    for (int k = 1; k <= n; k++)
        for (int i = 1; i <= n; i++) {
            if (d[i][k] >= INF) continue;          // optimizare
            for (int j = 1; j <= n; j++) {
                if (d[k][j] >= INF) continue;
                long alt = d[i][k] + d[k][j];
                if (alt < d[i][j]) d[i][j] = alt;
            }
        }
}

void init(int n, Alee *al, int m) {
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= n; j++)
            d[i][j] = (i == j) ? 0 : INF;
    for (int e = 0; e < m; e++)
        if (al[e].t < d[al[e].u][al[e].v])
            d[al[e].u][al[e].v] = al[e].t;
}
```

C2 – Detecția ciclurilor cu timp negativ

Cerința

Verificați dacă rețeaua conține un ciclu cu timp total negativ și, dacă da, raportați o piațetă care face parte dintr-un astfel de ciclu.

Indicii / discuție la tablă:

- După ce rulăm algoritmul de la C1, ce semnificație are valoarea $d[i][i]$? În condiții normale, ce valoare ar trebui să aibă?
- Ce înseamnă, intuitiv, dacă pentru o anumită piațetă i avem $d[i][i] < 0$ după rulare?
- De ce nu putem da încredere rezultatelor C3-C4 dacă există un astfel de ciclu?

C2 – Pseudocod și implementare

Idee: După Floyd-Warshall, $d[i][i] = 0$ în mod normal. Dacă $d[i][i] < 0$ pentru un i, înseamnă că există un ciclu cu timp total negativ care trece prin i – altfel cel mai scurt drum de la i la i ar fi cel "trivial" de lungime 0.

Listing 9: Pseudocod

```
pentru i = 1 .. n:
    daca d[i][i] < 0:
        returneaza (DA, i)
returneaza (NU, -1)
```

```
int ciclu_negativ(int n) {
    for (int i = 1; i <= n; i++)
        if (d[i][i] < 0) return i;      // piateta din ciclu
    return -1;                           // nu exista ciclu negativ
}
```

Complexitate: $O(n)$ după ce Floyd a fost deja rulat.

C3 – Locul de întâlnire optim

Cerința

Găsiți piațeta M care minimizează $\max(d(R, M), d(J, M))$ și timpul efectiv de întâlnire.

Indicii / discuție la tablă:

- De ce *maximul* și nu suma? Ambele personaje pleacă simultan; cine ajunge primul așteaptă pe cel rămas.
- Având deja matricea d de la C1, ce trebuie să facem pentru fiecare candidat M?
- Cum tratăm piațetele pentru care Romeo sau Julieta *nu pot* ajunge (drumul are valoare $+\infty$)?
- Care este complexitatea acestui pas, pe lângă cei $O(n^3)$ deja plătiți la C1?

C3 – Pseudocod și implementare

Idee: Iterăm prin toate cele n piațete; pentru fiecare, calculăm $\max(d[R][M], d[J][M])$; păstrăm minimul.

Listing 10: Pseudocod

```
best_M = -1; best_time = +INF
pentru M = 1 .. n:
    daca d[R][M] >= INF sau d[J][M] >= INF: continua
    t = max(d[R][M], d[J][M])
    daca t < best_time:
        best_time = t
        best_M = M
```

```
int loc_intalnire(int n, int R, int J, long *timp_out) {
    int best_M = -1;
    long best_time = INF;
    for (int M = 1; M <= n; M++) {
        if (d[R][M] >= INF || d[J][M] >= INF) continue;
        long t = (d[R][M] > d[J][M]) ? d[R][M] : d[J][M];
        if (t < best_time) { best_time = t; best_M = M; }
    }
    *timp_out = best_time;
    return best_M;
}
```

C4 – Reconstrucția traseelor

Cerința

Reconstruiți traseul efectiv al lui Romeo (de la R la M) și cel al Julietei (de la J la M).

Indicii / discuție la tablă:

- Algoritmul de la C1 *nu* reține drumurile, doar costurile. Ce informație trebuie să adăugăm pentru a putea reconstrui drumul?
- Două opțiuni clasice: matrice de *predecesori* (cine este penultimul pe drumul de la i la j) sau matrice de *succesori* (care este prima piațetă de pe drumul de la i la j). Care e mai naturală în contextul lui Floyd?
- Când relaxăm $d[i][j]$ prin intermediarul k, cum actualizăm matricea de succesori?

C4 – Pseudocod

Idee: Întreținem o matrice `urm[i][j]` = prima piatetă pe drumul minim de la i la j. Inițial `urm[u][v]` = v pentru fiecare alee. Când relaxăm prin k: noul drum de la i la j trece prin k – prima piatetă este aceeași cu prima de pe drumul de la i la k.

Listing 11: Pseudocod

```
init urm[i][j] = -1 pentru i != j; urm[i][i] = i
pentru fiecare alee (u, v, t):
    daca d[u][v] > t: d[u][v] = t; urm[u][v] = v

pentru k = 1 .. n:
    pentru i = 1 .. n, j = 1 .. n:
        daca d[i][k] + d[k][j] < d[i][j]:
            d[i][j]   = d[i][k] + d[k][j]
            urm[i][j] = urm[i][k]          // prima e pe drumul i -> k

afiseaza_drum(s, t):
    daca urm[s][t] == -1: "(nu exista)"; iesire
    afiseaza s
    cat timp s != t:
        s = urm[s][t]
        afiseaza " -> ", s
```

C4 – Implementare în C

```
int urm[NMAX][NMAX];

void init_cu_succesori(int n, Alee *al, int m) {
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= n; j++) {
            d[i][j]   = (i == j) ? 0 : INF;
            urm[i][j] = (i == j) ? i : -1;
        }
    for (int e = 0; e < m; e++) {
        int u = al[e].u, v = al[e].v;
        if (al[e].t < d[u][v]) {
            d[u][v]   = al[e].t;
            urm[u][v] = v;
        }
    }
}

void floyd_cu_succesori(int n) {
    for (int k = 1; k <= n; k++)
        for (int i = 1; i <= n; i++) {
            if (d[i][k] >= INF) continue;
            for (int j = 1; j <= n; j++) {
                if (d[k][j] >= INF) continue;
                long alt = d[i][k] + d[k][j];
                if (alt < d[i][j]) {
                    d[i][j]   = alt;
                    urm[i][j] = urm[i][k];
                }
            }
        }
}
```

C4 – Afișarea drumurilor

```c
void afiseaza_drum(int s, int t) {
    if (urm[s][t] == -1) {
        printf("(nu exista drum de la %d la %d)\n", s, t);
        return;
    }
    printf("  %d", s);
    while (s != t) {
        s = urm[s][t];
        printf(" -> %d", s);
    }
    printf("\n");
}

// Apel pentru Romeo si Julieta:
//   long tm;
//   int M = loc_intalnire(n, R, J, &tm);
//   printf("Loc intalnire: piateta %d, timp = %ld\n", M, tm);
//   printf("Drumul lui Romeo:\n");   afiseaza_drum(R, M);
//   printf("Drumul Julietei:\n");    afiseaza_drum(J, M);
```

Sinteză – ce am exersat astăzi

- **Problema 1 – Catalogul farmaciei**
  - Structură ordonată cu acces, inserare și ștergere în $O(h)$.
  - Cazul critic: ștergerea unui nod cu doi descendenți.
- **Problema 2 – Tastatura predictivă**
  - Structură arborescentă bazată pe litere, cu augmentare pentru "cel mai bun cuvânt" per subarbore.
  - Toate operațiile sunt $O(|\text{cuvânt}|)$, independent de mărimea vocabularului.
- **Problema 3 – Rețea de fibră**
  - Sortare după cost + Union-Find $\Rightarrow O(m \log m)$.
  - Traseul-strangulare este chiar cel mai scump traseu selectat.
- **Problema 4 – Romeo și Julieta**
  - Drumuri minime între toate perechile, cu suport pentru valori negative.
  - Detecția ciclurilor negative prin diagonala matricei.
  - Optimizare pentru întâlnire: $\min_M \max(d[R][M], d[J][M])$.

Mulțumesc!

Întrebări?

Universitatea Politehnica Timișoara
Departamentul de Calculatoare și Tehnologia Informației